How Claude Code Works

深入解读当前最成功的 AI 编程 Agent 的源码架构

stars 1.2k | forks 390 | License MIT | Source TypeScript | Docs 12 chapters

📘 在线阅读文档 → | English

> 🛠️ **想动手造一个？** 配套项目 Claude Code From Scratch — ~3000 行 TypeScript，11 章分步教程，从零构建你自己的 Claude Code

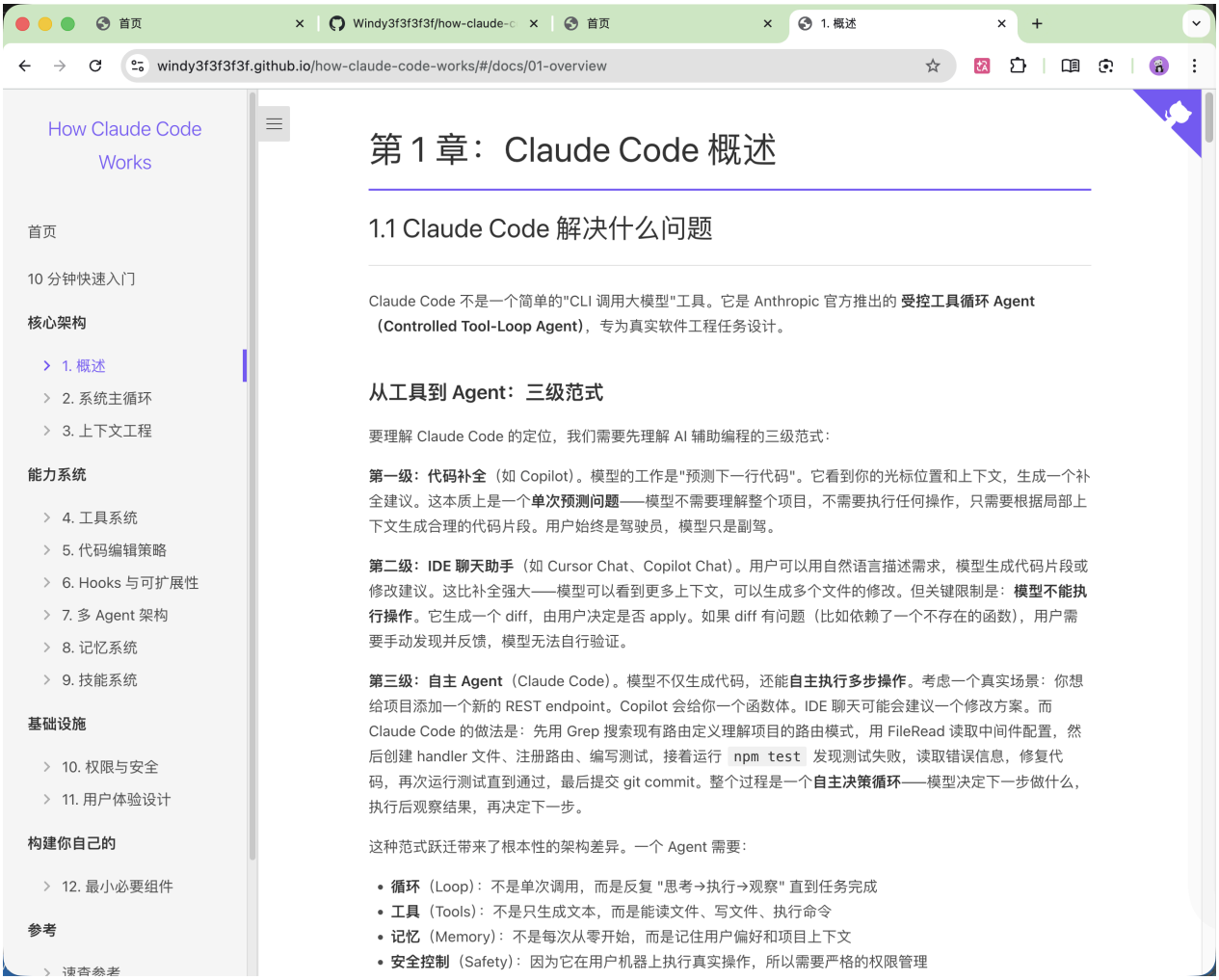
---

Claude Code 是目前使用最广泛的 AI 编程 Agent，也是我认为最好用的AI编程工具（我使用Claude Code已经一年了）。它能理解整个代码仓库、自主执行多步编程任务、安全地运行命令——而这一切背后是 **50 万行 TypeScript 源码**中沉淀的工程智慧。

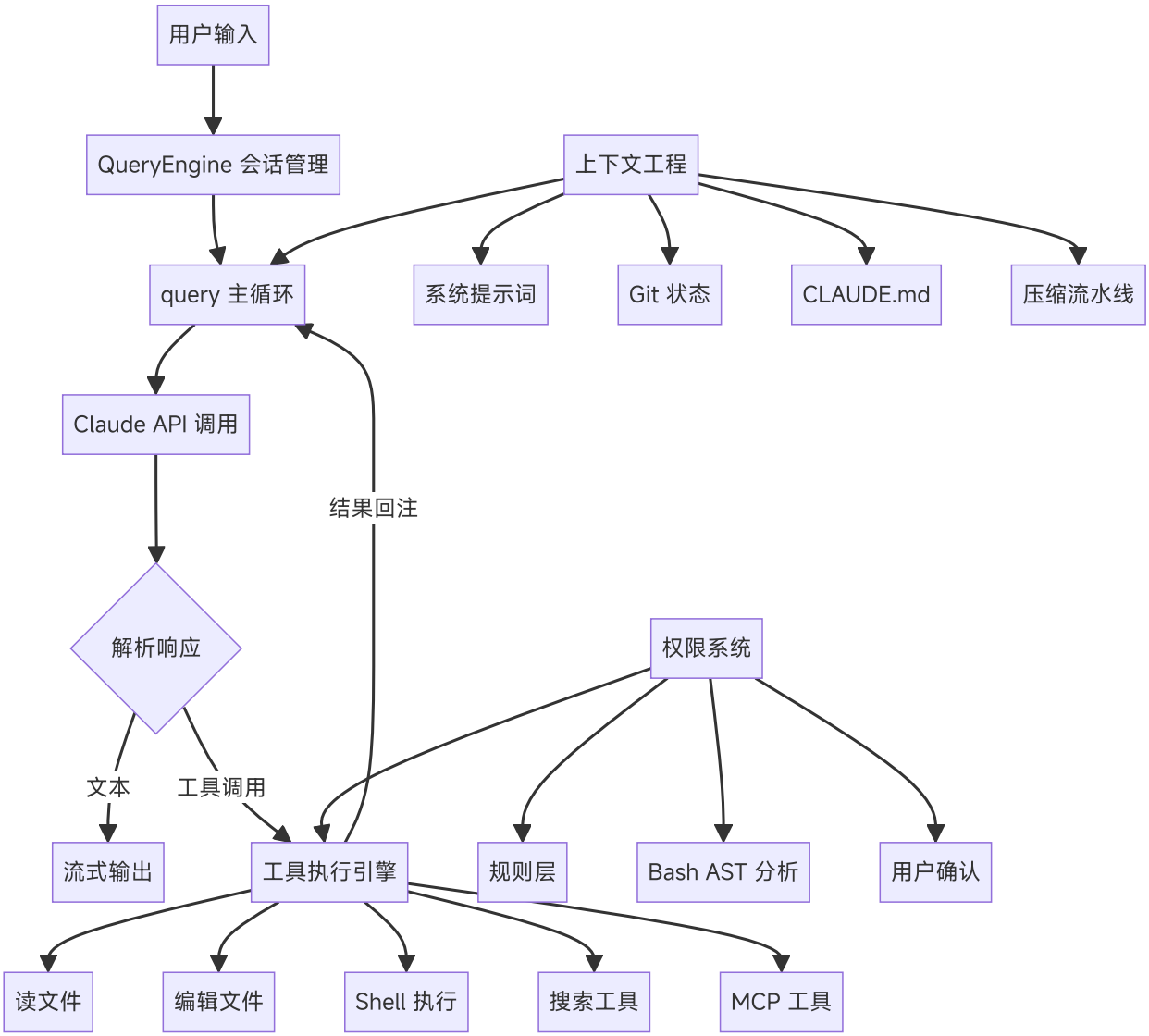
Anthropic 开源（x）了这份源码。**但 50 万行代码，从哪里开始读？**

这也是我创建这个项目的初衷，我自己也遇到了没有办法阅读这么长的代码项目的问题，我的解决方案是和Claude Code一起读，让他给我写文档配合我读源代码。在此同时，我想把这个过程文档化，就形成了这个项目。

我和Claude Code加班从源码中提炼出 **12 篇专题文档**（共33.8w字符），覆盖了从核心循环到安全防护的每一个关键设计决策。不管你是想造自己的 AI Agent，还是想更深入地理解和使用Claude Code，这里都是最短路径（应该？就算不是最短的，我也会不断更新这个项目）。


首页
Windy3f3f3f3f/how-claude-c
首页
1. 概述
windy3f3f3f3f.github.io/how-claude-code-works/#/docs/01-overview
How Claude Code Works
首页
10 分钟快速入门
核心架构
1. 概述
2. 系统主循环
3. 上下文工程
能力系统
4. 工具系统
5. 代码编辑策略
6. Hooks 与可扩展性
7. 多 Agent 架构
8. 记忆系统
9. 技能系统
基础设施
10. 权限与安全
11. 用户体验设计
构建你自己的
12. 最小必要组件
参考
第 1 章： Claude Code 概述
1.1 Claude Code 解决什么问题
Claude Code 不是一个简单的"CLI 调用大模型"工具。它是 Anthropic 官方推出的 受控工具循环 Agent（Controlled Tool-Loop Agent），专为真实软件工程任务设计。
从工具到 Agent：三级范式
要理解 Claude Code 的定位，我们需要先理解 AI 辅助编程的三级范式：
第一级：代码补全（如 Copilot）。模型的工作是"预测下一行代码"。它看到你的光标位置和上下文，生成一个补全建议。这本质上是一个单次预测问题——模型不需要理解整个项目，不需要执行任何操作，只需要根据局部上下文生成合理的代码片段。用户始终是驾驶员，模型只是副驾。
第二级：IDE 聊天助手（如 Cursor Chat、Copilot Chat）。用户可以用自然语言描述需求，模型生成代码片段或修改建议。这比补全强大——模型可以看到更多上下文，可以生成多个文件的修改。但关键限制是：模型不能执行操作。它生成一个 diff，由用户决定是否 apply。如果 diff 有问题（比如依赖了一个不存在的函数），用户需要手动发现并反馈，模型无法自行验证。
第三级：自主 Agent（Claude Code）。模型不仅生成代码，还能自主执行多步操作。考虑一个真实场景：你想给项目添加一个新的 REST endpoint。Copilot 会给你一个函数体。IDE 聊天可能会建议一个修改方案。而 Claude Code 的做法是：先用 Grep 搜索现有路由定义理解项目的路由模式，用 FileRead 读取中间件配置，然后创建 handler 文件、注册路由、编写测试，接着运行 npm test 发现测试失败，读取错误信息，修复代码，再次运行测试直到通过，最后提交 git commit。整个过程是一个自主决策循环——模型决定下一步做什么，执行后观察结果，再决定下一步。
这种范式跃迁带来了根本性的架构差异。一个 Agent 需要：
• 循环（Loop）：不是单次调用，而是反复 "思考→执行→观察" 直到任务完成
• 工具（Tools）：不是只生成文本，而是能读文件、写文件、执行命令
• 记忆（Memory）：不是每次从零开始，而是记住用户偏好和项目上下文
• 安全控制（Safety）：因为它在用户机器上执行真实操作，所以需要严格的权限管理

🏗️ 系统架构



🤔 这份源码为什么值得深入研究?

大多数 AI Agent 框架都是"demo 级别"——跑通一个场景就宣布成功。Claude Code 不同，它是一个**日活百万级开发者实际使用的生产系统**，需要处理的问题远比 demo 复杂：

- 对话动辄上百万 token，上下文窗口不够用怎么办？（记忆管理、压缩方案是如何设计的超重要）
- 66 个内置工具同时存在，怎么协调？（如果所有工具上下文都给AI，那将直接爆炸）
- 怎么让用户感觉"快"，哪怕模型推理本身就要几十秒？（如何实现流水线设计）
- 用户让 AI 执行 `rm -rf /`，怎么拦住？（安全护栏很重要）

这些问题的解法，就藏在源码里。

🔍 从源码中发现的关键设计

> 以下内容均来自对源码的实际分析，不是猜测。

为什么 Claude Code 用起来感觉那么快？

它其实做了三件聪明的事：

1. **全链路流式输出** — 不是等模型全部想完再显示，而是每生成一个 token 就立刻展示。从 API 调用到终端渲染，整条链路都是流式的。
2. **工具预执行** — 模型说"我要读某个文件"的时候，这个文件其实已经在读了。系统在模型还在输出的同时就开始解析和执行工具调用，利用模型生成的 5-30 秒窗口，把约 1 秒的工具延迟藏起来了。
3. **9 阶段并行启动** — 启动时把不相关的初始化任务并行执行，关键路径压到约 235ms。

出错了怎么办？—— 静默恢复

普通程序遇到错误会报错给用户。Claude Code 的策略是：**能恢复的错误，用户根本看不到。**

比如对话太长超出了上下文窗口，它不会弹个错误框让你手动处理，而是悄悄压缩上下文、自动重试。token 输出达到上限？自动从 4K 升级到 64K 再重试。整个 Agent 循环有 7 种不同的"继续"策略，每种对应一种故障恢复路径。

这就是为什么用 Claude Code 的时候很少遇到报错——不是没有错误，而是大部分都被内部消化了。

对话太长怎么办？—— 4 级渐进式压缩

这是整个系统中最精妙的设计之一。当上下文快要超限时，不是一刀切地压缩，而是分 4 个级别逐步处理：

1. **裁剪** — 先把历史消息中的大块内容（旧的工具输出）截断
2. **去重** — 几乎零成本地去除重复内容
3. **折叠** — 把不活跃的对话段落折叠起来，但不修改原始内容（可以展开恢复）
4. **摘要** — 最后手段，启动一个子 Agent 对整个对话做摘要

每一级都可能释放足够的空间，让后面的级别不需要执行。而且压缩后系统会**自动恢复最近编辑的 5 个文件内容**，防止模型忘记刚刚在干什么。

怎么防止 AI 执行危险操作？—— 5 层纵深防御

Claude Code 让 AI 直接在你电脑上跑命令，安全设计必须过硬。它不是靠一个"你确定吗？"对话框，而是搭建了 5 层防御体系：

1. **权限模式** — 不同信任级别，限制可执行的操作范围
2. **规则匹配** — 基于命令模式的白名单/黑名单
3. **Bash 命令深度分析** — 这里最硬核：用语法树分析（不是正则匹配）拆解 Shell 命令的真实意图，包含 23 项安全检查，覆盖命令注入、环境变量泄露、特殊字符攻击等
4. **用户确认** — 危险操作弹出确认对话框，但有 200ms 防抖保护，防止键盘连击导致误确认
5. **Hook 校验** — 允许用户自定义安全规则，甚至可以动态修改工具的输入参数（比如自动给 `rm` 加上 `--dry-run`）

这五层任何一层拦住就不会执行，纵深防御。

66 个工具如何协同工作？

所有工具——读文件、写文件、跑命令、搜索、甚至第三方 MCP 工具——都遵循**同一套接口规范**。这意味着：

- 第三方工具和内置工具走完全相同的执行流水线，享受同样的安全检查和权限控制
- 只读工具自动并行执行，写操作自动串行，不需要手动管理并发
- 工具输出超过 100K 字符时自动落盘，模型只拿到摘要和文件路径，需要时再读取全文

多个 Agent 如何协作？

Claude Code 支持三种多 Agent 模式：

- **子 Agent** — 主 Agent 分派任务给子 Agent，等结果返回
- **协调器** — 纯指挥官模式，协调器只能分配任务，**不能自己读文件、写代码**，强制分工
- **Swarm** — 多个命名 Agent 之间点对点通信，各自独立工作

为了防止多个 Agent 同时改同一个文件产生冲突，系统用 Git Worktree 给每个 Agent 一份独立的代码副本。

📚 专题深入

#	文档	你会了解到
1	概述	技术选型背后的思考（为什么 Bun/React/Zod）、6 条核心设计原则、9 阶段 235ms 启动流程、数据流全景

#	文档	你会了解到
2	系统主循环	Agent 循环的双层架构、7 种 Continue Sites 故障恢复、工具预执行、StreamingToolExecutor 并发机制
3	上下文工程	4 级压缩流水线完整细节、压缩后自动恢复机制（5 文件 + 技能重激活）、提示词缓存策略与缓存断裂检测
4	工具系统	66 个工具的注册与并发控制、MCP 7 种传输详解、连接状态机、OAuth 2.0 + PKCE 认证流程
5	代码编辑策略	search-and-replace 为什么比整文件重写更好、唯一性约束与抗幻觉设计、编辑前强制读取的代码级实现
6	Hooks 与可扩展性	23+ Hook 事件全景、5 种 Hook 类型、6 阶段执行管道、PermissionRequest 4 种能力、信任模型与安全
7	多 Agent 架构	子 Agent 4 种执行模式与 Worktree 隔离、协调器纯编排设计、Swarm 3 种执行后端与信箱通信
8	记忆系统	4 种记忆类型与封闭分类法、Sonnet 语义召回与异步预取、后台记忆提取 Agent、记忆漂移防御、团队记忆
9	技能系统	6 层技能来源与优先级、懒加载与 Token 预算分配、Inline/Fork 双执行模式、白名单权限模型、压缩后技能保留
10	权限与安全	5 层纵深防御体系、tree-sitter AST 分析 + 23 项安全检查、竞速确认机制与 200ms 防误触
11	用户体验设计	自研 Ink 渲染器架构、Yoga Flexbox 布局、虚拟滚动与对象池优化、Vim 模式
12	最小必要组件	7 个最小必要组件框架、最小实现 vs 生产级实现的逐项对照、从 500 行到 50 万行的演进路线

🎯 谁应该读这个?

你是	你能获得
想做 AI Agent 产品的开发者	一个经过百万用户验证的架构参考，少走弯路
Claude Code 用户	理解它为什么这样工作，学会用 Hooks 和 CLAUDE.md 深度定制
对 AI 安全感兴趣的人	生产级 AI 系统的安全设计实战，不是论文里的理论
学生或 AI 研究者	大规模工程实践的第一手材料，比任何教科书都真实

📊 关键数据

指标	数值
源码总行数	512,000+
TypeScript 文件	1,884
内置工具	66+
压缩流水线级数	4 级
权限防御层数	5 层

🗺️ 阅读建议

只有 10 分钟？ → 读 快速入门

想理解核心原理？ → 按顺序读 主循环 → 上下文工程 → 工具系统

想自己造一个 AI Agent？ → 先读 最小必要组件，然后跟着 claude-code-from-scratch 的 11 章教程动手实现——~3000 行代码，每一步都对照源码讲解

想定制 Claude Code？ → 读 Hooks 与可扩展性 + 记忆系统 + 技能系统

关注安全？ → 读 权限与安全 + 代码编辑策略

🤝 贡献

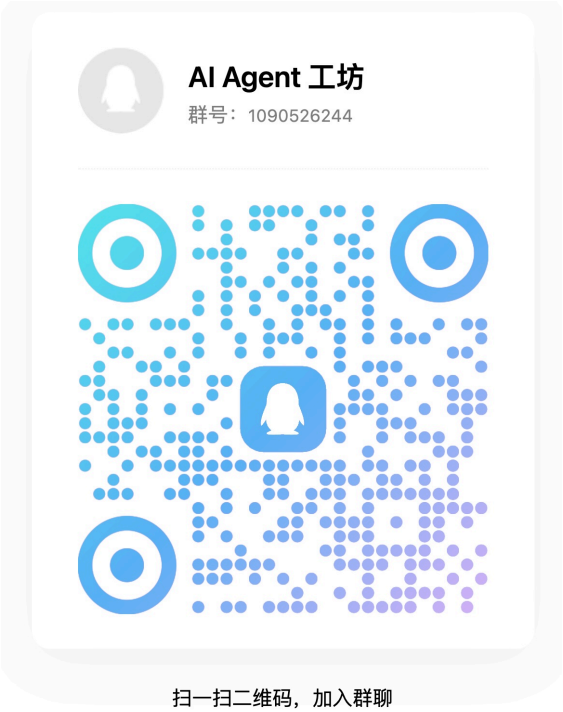
欢迎提 Issue 和 PR！如果你发现分析有误或有更好的理解角度，非常欢迎讨论。

🙏 致谢

感谢 LINUX DO 社区的支持与讨论。

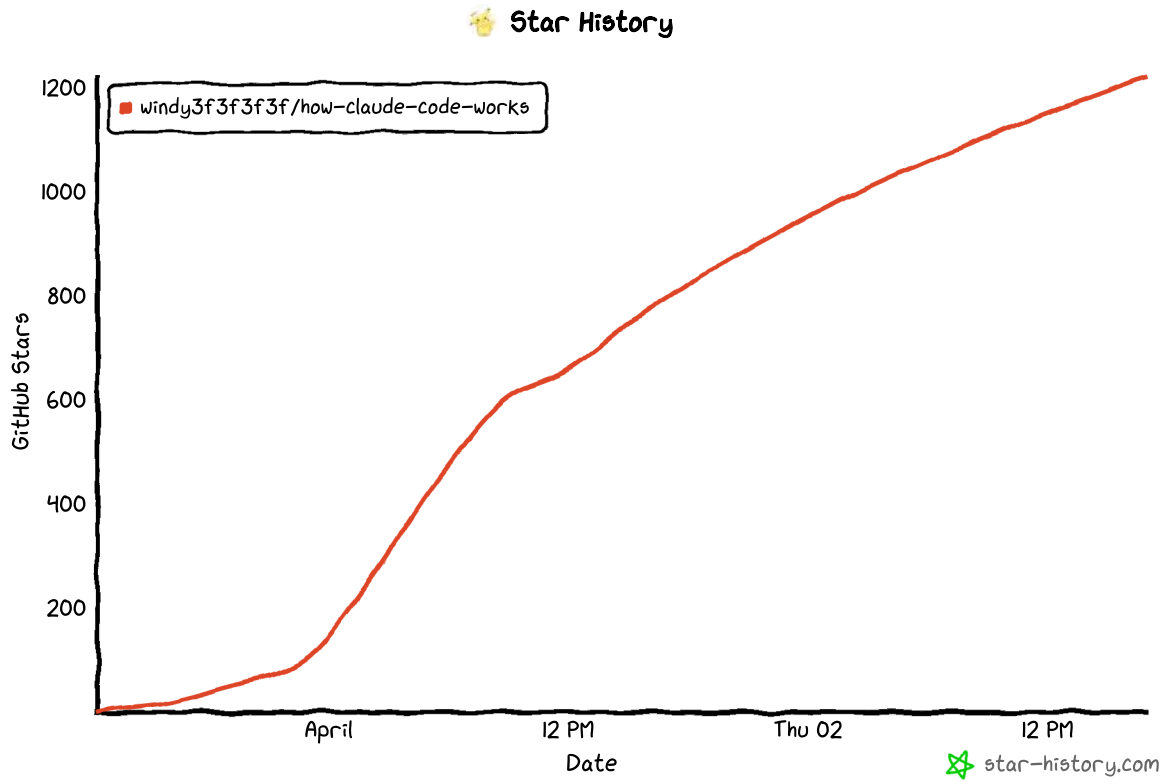
💬 更多交流

加入 AI Agent 工坊 交流群



QQ 群号：**1090526244**

📈 Star History



📝 更新记录

日期	更新内容
2026-04-01	拆分记忆与技能为独立章节（11→12 篇），按侧边栏分组重新编号 01-12
2026-04-01	全部 12 章大幅扩充（篇幅翻倍），补充源码级实现细节、Mermaid 架构图、代码示例
2026-04-01	同步更新 quick-start 总览页、README 文档目录描述，匹配各章节新增内容
2026-03-31	新增 3 章：Hooks 与可扩展性、多 Agent 架构、记忆与技能系统
2026-03-31	充实原有 8 章内容，补充启动流程、MCP 集成、压缩恢复机制等专题
2026-03-31	上线 Docsify 文档站点，支持搜索、Mermaid 渲染、章节导航

日期	更新内容
2026-03-31	初始发布：8 篇核心架构分析文档

📄 License

MIT